Internal Module and Data Contracts

Media Reminder - Offline-First Adaptive Media Alarm

Prepared for Mohamed Aslam Abdul

2026-08-05

Contents

0.1 Document control

Field	Value
Document ID	MR-08
Version	1.0
Status	Approved baseline
Last updated	2026-08-05
Product owner	Mohamed Aslam Abdul
Package identifier	`com.aslam.mediareminder`
Purpose	Define typed React Native/native boundaries, domain commands, data transfer objects, events, error codes and compatibility rules.

> **Reading rule:** This pack specifies a production-oriented Android application, not a promise that third-party devices will behave identically. Where Android or an OEM controls presentation, timing, sound, or permissions, the app provides transparent status and the strongest compliant fallback.

0.2 Document conventions

The words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, and **MAY** are normative. A requirement ID is stable once published. Requirement IDs may be retired but MUST NOT be reassigned to a different meaning.

Term	Meaning
Reminder	A user-authored instruction that connects content, a schedule, and a presentation profile.
Occurrence	One calculated due instance of a reminder.
Alarm session	The bounded runtime state created when an occurrence is actively alerting.

Term	Meaning
Media asset	An app-owned local video, audio, image, or text item.
Profile	Reusable alert behavior such as Gentle, Standard, or Persistent.
Exact-alarm access	Android special app access that allows exact scheduling where the platform requires it.
Full-screen intent	Android notification mechanism for urgent, time-sensitive activity presentation. It is not a general overlay.
Heads-up notification	System-rendered high-priority notification shown temporarily over the current app when Android permits it.
Source of truth	The authoritative specification or local persisted record for a decision or state.

1 Contract principles

Contracts are versioned, narrow and use-case oriented. UI code does not receive Room entities, file paths, Android URIs, `PendingIntent`s or platform objects. UUIDs are lowercase canonical strings. Instants are UTC ISO-8601 strings. Local times use `HH:mm:ss` plus IANA zone identifiers where required.

2 Native module surface

Conceptual Codegen interface:

```
export interface Spec extends TurboModule {
  getStartupSnapshot(): Promise<StartupSnapshot>;
  listMedia(query: MediaQuery): Promise<Page<MediaSummary>>;
  getMedia(id: UUID): Promise<MediaDetail>;
  beginMediaImport(request: ImportRequest): Promise<OperationRef>;
  updateMedia(request: UpdateMediaRequest): Promise<MediaDetail>;
  deleteMedia(request: DeleteMediaRequest): Promise<MutationResult>;

  listReminders(query: ReminderQuery): Promise<Page<ReminderSummary>>;
  getReminder(id: UUID): Promise<ReminderDetail>;
  saveReminder(request: SaveReminderRequest): Promise<SaveReminderResult>;
  setReminderEnabled(id: UUID, enabled: boolean): Promise<EnableResult>;
  deleteReminder(id: UUID): Promise<MutationResult>;

  listProfiles(): Promise<ReminderProfile[]>;
  saveProfile(request: SaveProfileRequest): Promise<ReminderProfile>;
  resetBuiltInProfile(id: UUID): Promise<ReminderProfile>;

  getCapabilitySnapshot(): Promise<CapabilitySnapshot>;
  openCapabilitySettings(kind: CapabilityKind): Promise<LaunchResult>;
  scheduleTestReminder(mode: TestMode): Promise<TestReminderResult>;

  beginExport(request: ExportRequest): Promise<OperationRef>;
  inspectBackup(uriToken: string): Promise<BackupInspection>;
  commitImport(request: CommitImportRequest): Promise<ImportResult>;
  cancelOperation(id: UUID): Promise<CancelResult>;

  playDueSession(sessionId: UUID, nonce: string): Promise<ActionResult>;
  snoozeDueSession(sessionId: UUID, minutes: number, nonce: string): Promise<ActionResult>;
```

```
  dismissDueSession(sessionId: UUID, nonce: string): Promise<ActionResult>;
}
```

The exact generated syntax follows the installed React Native Codegen version. Semantic names and payload rules are binding.

3 Shared primitives

```
type UUID = string;
type Instant = string;        // UTC, e.g. 2026-08-05T22:15:00Z
type LocalDate = string;       // YYYY-MM-DD
type LocalTime = string;       // HH:mm:ss
type ZoneId = string;          // IANA identifier
type CorrelationId = string;

type ResultStatus = 'ok' | 'limited' | 'needs_action';
```

Numbers representing byte sizes are decimal strings in bridge DTOs when they may exceed JavaScript's safe integer range. Durations are integer milliseconds only where bounded below `Number.MAX_SAFE_INTEGER`; schedule snooze values use integer minutes.

4 Startup snapshot

```
interface StartupSnapshot {
  contractVersion: 1;
  schemaVersion: number;
  appVersion: string;
  mediaCount: number;
  activeReminderCount: number;
  nextOccurrence: OccurrenceSummary | null;
  capability: CapabilitySnapshot;
  repair: RepairSummary;
  activeSession: ActiveSessionSummary | null;
  sequence: string;
}
```

A contract version mismatch is a hard developer error in debug and a user-safe update-required screen in release. Additive optional fields are backward compatible; renamed or semantic changes require a new contract version.

5 Media DTOs

```
type MediaKind = 'video' | 'audio' | 'image' | 'text';
type IntegrityState = 'healthy' | 'unchecked' | 'missing' | 'changed' | 'unsupported';

interface MediaSummary {
  id: UUID;
  kind: MediaKind;
  title: string;
  durationMs?: number;
  sizeBytes: string;
  thumbnailToken?: string;
  category?: NamedRef;
  tags: NamedRef[];
  activeReminderCount: number;
  integrity: IntegrityState;
  createdAt: Instant;
}
```

`thumbnailToken` is an opaque app-local token consumed by an image provider; it is not an absolute path. UI sends an import intent request and native code launches the system picker so URI lifecycle stays native and typed.

6 Reminder DTOs

```ts
interface ReminderDraft {
  id?: UUID;
  mediaId: UUID;
  label: string;
  notes?: string;
  schedule: ScheduleRuleDto;
  profileId: UUID;
  snooze: SnoozePolicyDto;
  enabledIntent: boolean;
}

type ScheduleRuleDto =
  | { type: 'once'; instant: Instant; originZone: ZoneId }
  | { type: 'daily'; localTime: LocalTime; zonePolicy: 'follow_device' }
  | { type: 'weekdays'; localTime: LocalTime; isoWeekdays: number[]; zonePolicy: 'follow_device' };

interface SaveReminderResult {
  reminder: ReminderDetail;
  nextOccurrence: OccurrenceSummary | null;
  capabilityResult: CapabilityEvaluation;
  schedulerGeneration: string;
}
```

UI never calculates authoritative next occurrence. It may preview locally for responsiveness, but Save returns native-calculated truth and any DST resolution.

7 Capability contract

```ts
type CapabilityKind =
  | 'notifications'
  | 'exact_alarm'
  | 'full_screen_intent'
  | 'channels'
  | 'battery_environment'
  | 'scheduler';

interface CapabilityItem {
  kind: CapabilityKind;
  status: 'ready' | 'limited' | 'blocked' | 'unknown';
  effectKey: string;
  action: 'none' | 'request_runtime' | 'open_special_access' | 'open_channel' | 'run_test' |
'open_app';
  observedAt: Instant;
}
```

The bridge exposes semantic status, not raw permission constants. Raw platform values may be included in diagnostic-only fields unavailable to normal UI.

8 Operations and progress

Long-running imports/exports are represented by operation UUID. Progress events:

```ts
interface OperationProgressEvent {
  operationId: UUID;
  kind: 'import' | 'export' | 'backup_inspection' | 'backup_commit' | 'repair';
  phase: string;
  completedUnits?: string;
  totalUnits?: string;
  currentItemIndex?: number;
  totalItems?: number;
  cancellable: boolean;
  sequence: string;
  correlationId: CorrelationId;
}
```

Events are hints. A screen restored after process death calls `getOperation(id)` or startup snapshot; it does not assume it received every event.

9 Alarm action contract

Action nonce is generated by native notification/activity code. A result may be replayed for an identical nonce.

```
interface ActionResult {
  sessionId: UUID;
  outcome: 'playing' | 'snoozed' | 'dismissed' | 'already_resolved' | 'media_unavailable' |
'failed_safe';
  effectiveAt: Instant;
  snoozedUntil?: Instant;
  nextOccurrence?: OccurrenceSummary;
}
```

Native receivers call the same use case directly; the React bridge is not in that path.

10 Backup contract

```
interface BackupInspection {
  archiveVersion: string;
  createdAt: Instant;
  sourceAppVersion: string;
  mediaCount: number;
  reminderCount: number;
  compressedBytes: string;
  expectedUncompressedBytes: string;
  checksumStatus: 'valid' | 'invalid' | 'missing';
  compatibility: 'compatible' | 'migratable' | 'too_new' | 'unsupported';
  conflicts: ConflictSummary[];
  warnings: string[];
  importToken: string;
}
```

`importToken` references validated private staging and expires. `CommitImportRequest` cannot accept a raw URI; it must use the token from inspection so commit never bypasses validation.

11 Error envelope

Rejected promises use a normalized object:

```
interface NativeErrorEnvelope {
  code: string;
  messageKey: string;
  category: 'validation' | 'capability' | 'storage' | 'media' | 'schedule' | 'backup' | 'security' |
'internal';
  retryable: boolean;
  correlationId: CorrelationId;
  field?: string;
  safeDetails?: Record<string, string | number | boolean>;
}
```

Representative codes:

Code	Meaning	Retry
`MR_MEDIA_UNSUPPORTED_TYPE`	Header/probe not accepted	No, choose another file
`MR_STORAGE_INSUFFICIENT`	Free space below required reserve	After freeing space
`MR_SCHEDULE_NONEXISTENT_TIME`	DST gap requires resolution	After user choice
`MR_EXACT_ACCESS_REQUIRED`	Exact mode selected but access missing	After settings or Limited choice
`MR_NOTIFICATION_BLOCKED`	Alert cannot be shown	After permission/channel

Code	Meaning	Retry
		change
`MR_BACKUP_CHECKSUM_INVALID`	Archive content mismatch	No; use another archive
`MR_BACKUP_TOO_NEW`	Reader lacks schema migrator	After app update
`MR_ACTION_ALREADY_RESOLVED`	Duplicate action	Treated as success-like result
`MR_INTERNAL_FAILED_SAFE`	Invariant failed; resources stopped	Diagnostic/retry depending context

Messages are localized by key in React Native or native resources. Error envelopes MUST NOT contain file paths, labels or media titles.

12 Versioning rules

- Bridge contract version is integer and changes only for breaking semantics.
- Database schema version follows Room migration numbering.
- Backup archive version uses semantic `major.minor`.
- Domain event/diagnostic version is independent.
- Every release records all four versions in About and diagnostic export.

An older UI bundle must never run against a newer incompatible native contract. Standard packaged React Native builds prevent this, but OTA updates are not used in v1, eliminating split-version risk.

13 Event ordering and idempotency

Each event stream has a monotonically increasing sequence persisted per process/session as appropriate. UI compares sequence and refetches after gaps. Mutating requests may include a client operation ID; native stores recent IDs and returns the original result within the idempotency window.

Alarm actions require native-generated nonce and session. Backup commit requires validated import token. Delete operations require current entity version to prevent overwriting concurrent changes.

14 Security properties

- No method returns arbitrary filesystem paths.
- No method opens arbitrary intent URI supplied by JavaScript without validation.
- All share operations use explicit read-only grants.
- Sensitive media is not serialized across the bridge.
- Debug-only injection methods are absent from release Codegen spec.
- Every exported Android component validates action, package, token and record state.

15 Contract test requirements

Golden JSON fixtures exist for every DTO and error. TypeScript tests decode native fixtures; Kotlin tests encode the same semantic objects. Breaking changes fail CI unless contract version and migration guide

are updated. Fuzz tests cover unknown fields, missing optional fields, invalid enums, oversized strings and malicious backup DTO values.

15.1 Governance

This document is part of the **Media Reminder Source-of-Truth Pack v1.0**. In case of conflict, apply this precedence order: (1) explicit platform safety and permission rules, (2) Architecture Decision Records, (3) Android specification, (4) data and backup specifications, (5) PRD and feature specification, (6) UX and visual guidance, (7) implementation notes. Record any intentional deviation in the decision log before release.